

《UML2 面向对象分析与设计》综合案例：员工考勤系统

作业评分实施细则

一、第四章作业（用例图和用例文档）

1. 评分档次

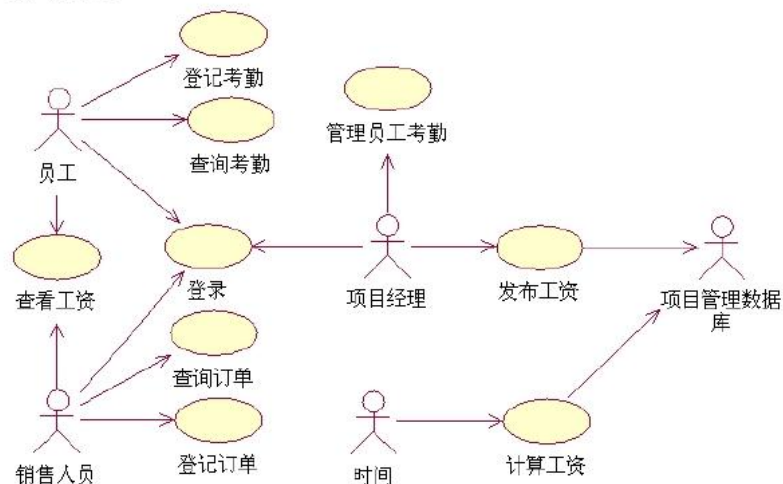
用例图和用例文档分别按照满分 10 分计算，以此作为评分标准，基本的评分准则如下：

- 一档（10 分）：图形（文本）条理清楚，无任何明显错误
- 二档（8-9 分）：图形/文本清楚，存在个别错误
- 三档（6-7 分）：图形/文本一般，存在一定的错误
- 四档（5 分）：图形/文本条理不清，存在致命错误或错误数过多

一般情况下按错别个数扣分，每个错误按严重程度扣 0.5、1、2 分，最终成绩向上取整；同类错误不重复扣分。

2. 参考答案

作业答案部分仅供参考，学生的作业可能会多种多样，具体按照第三部分的典型错误扣分，用例图：



用例文档：

用例编号：UC02	用例名：登记考勤	作者：谭火彬
用例描述：小时工和普通员工通过该用例来编辑和查看自己的考勤信息		
参与者		

员工（含小时工和普通员工）		
相关用例	无	
前置条件	员工已正确登录到该系统	
后置条件	无（将在下次迭代中确定）	
涉众利益	员工：准确地维护自己的考勤信息 公司：要求员工的信息准确	
基本路径	1—添加新的考勤 1.1、 用例起始于用户需要记录新的考勤信息 1.2、 系统显示当前日期和时间，并提醒用户该时间即为用户的上班時間 1.3、 用户确认该信息 1.4、 系统记录当前日期和时间，并将其作为用户考勤信息的上班時間 2—提交考勤信息 2.1、 任何时刻用户都可以提交自己的考勤信息 2.2、 系统查询用户上班时的考勤记录（E-1） 2.3、 系统记录当前的日期和时间，作为用户考勤信息的下班時間 2.4、 系统显示用户今天完整的考勤信息 2.5、 用户确认提交考勤信息 2.6、 系统保存考勤信息，并将考勤信息的状态改为“已提交”（D-1）	
备选路径	E-1 如果系统没有找到用户上班时的考勤信息，则用例终止；用户可以通过项目经理为其添加上班的考勤信息	
数据需求	A-1 考勤信息主要包括：用户名、日期、上班时间、下班時間、状态 D-1 考勤信息的状态有：“新考勤”（只有上班时间，没有下班時間的考勤信息）、“已提交”（有完整的上下班時間，但还没有进行工资结算的考勤）、“已完成”（已结算工资的考勤）	
业务规则	B-1 作为用户考勤信息的上下班時間由系统自动获取，不允许用户编辑 B-2 状态为“已提交”的考勤信息不允许普通用户进行任何操作；	
非功能需求	无	
设计约束	无	
待解决问题	无	

用例编号： UC06	用例名： 计算工资	作者： 谭火彬
用例描述： 该用例由系统定期启动，根据员工的考勤和工资级别等信息计算各类员工的工资		

参与者	时间、项目管理数据库（外部系统）
相关用例	无
前置条件	无
后置条件	无（将在下次迭代中确定）
涉众利益	员工：...（包括临时工、普通员工、销售人员） 公司：...
基本路径	—计算普通员工和销售员工工资 1. 用例起始于系统时间到达每月末晚上，需要计算普通员工和销售员工工资(E-1)； 2. 系统查询所有的普通员工和销售人员的个人信息（D-1）； 3. 对于每一个员工（普通员工、销售人员）： 3.1. 根据员工的类别获得其考勤信息或订单信息（E-2）； 3.1.1. 如果是普通员工，则获得本月的考勤信息（D-2）； 3.1.2. 如果是销售人员，则获得本月的销售信息（D-3）； 3.2. 系统从项目管理数据库中获得员工的工资级别信息（E-3）； 3.3. 系统根据员工的考勤信息（或销售信息）和工资级别信息计算该员工的工资，保存； 4. 计算完成后，系统产生一个提醒信息，以便于项目经理确认
备选路径	E-1—计算临时工工资 1. 用例起始于系统时间达到每个周末的晚上，需要计算临时工工资 2. 系统查询所有临时工的个人信息 3. 对于每一个临时工： 3.1. 获得员工的考勤信息 3.2 从项目管理数据库中获得员工的工资级别信息； 3.3 系统根据员工的考勤信息和工资级别信息计算该员工的工资，保存； 4. 计算完成后，系统产生一个提醒信息，以便于项目经理确认 E-2 如果找不到该员工的考勤信息或订单信息，则记录相关日志，并转回 3 计算下一个员工 E-3 如果无法获得员工工资级别信息，则记录相关日志，并转回 3 计算下一个员工
数据需求	D-1. 员工信息=员工编号+员工姓名 D-2 考勤信息参见“登记考勤”用例 D-3 订单信息参见“登记订单”用例
业务规则	暂不明确
非功能需求	暂不明确
设计约束	

无
待解决问题 1. 系统具体的运行日期和时间有待和用户进一步商定 3.2 员工的工资级别信息的具体项目有待进一步明确 3.3 详细的工资计算细节有待进一步明确
相关图 (可画该用例的活动图)

3. 典型错误情况

3.1 用例图部分

3.1.1 参与者

本系统中包含的参与者有：小时工、普通员工、销售人员、项目经理、项目管理数据库、时间，其中由于小时工和普通员工有关考勤的处理细节完全相同，因此为了便于简化和复用，可将他们统一合并为员工（不合并也可以，不算错误），但不能和销售人员合并，因为销售人员没有考勤信息，而是登记订单信息，需要明确区分。

(1) 时间参与者的使用

时间参与者，一种习惯用法，用于激活那些系统定期的、自动执行的用例；它只是一种抽象，在定义参与者是不应该将其具体化。如在本系统中存在每周五、月末这样的具体时间点，这些细节不需要在用例图中表现，如有存在，扣 0.5-1 分；

(2) 外部系统参与者

本系统需要和现有的项目管理数据库进行交互，因此该数据库是一个外部系统，需要定一个外部系统参与者“项目管理数据库”，注意此处应从用户角度定义参与者的名字，不能使用“数据库（扣 1 分）”、或“DB2 数据库/系统”（扣 0.5 分）；

(3) “人”作为参与者

“人”作为一个外部因素，当他直接与系统交互时，在用例模型中作为一个参与者表示，但系统不能对他提出任何要求（即人不可能实现系统职责），因此注意用例与“人”关联的方向，一般情况下不要划由用例指向人的方向的关联（扣 0-0.5 分）；

(4) 参与者泛化关系的使用

泛化代表责任重叠，如果两个参与者之间存在泛化关系，则特化的参与者会继承泛化参与者的所有用例，因此特化参与者不能再和泛化参与者相关用例再建立关联（扣 0-0.5 分）；

此外，在定义泛化关系时要明确，特化的参与者需要泛化参与者所提供的功能，如本系统中项目经理和员工之间可以不定义泛化关系，因为需求中并没有明确表明项目经理需要进行考勤等员工相关操作。

最后，如果某泛化/特化的参与者没有参与任何用例，则该参与者可以省略；注意参与者之间只有泛化关系，不存在其它的任何关系（扣 0.5-1 分）。

3.1.2 用例

本系统中的用例有：登记考勤、查询考勤（这两个可合并成一个维护个人考勤）、查看工资、登记订单、查询订单（这两个也可合并成维护个人订单）、管理员工考勤、计算工资、

发布工资和登录；具体细节参看用例图。

(1) 用例的命名

用例的命名按照“(状语)动词+(定语+)宾语”，且名称要简单易懂，直接反映业务本质，少用“信息”、“数据”等弱名次。典型的错误有：

“考勤信息”：缺少动词，到底如何处理考勤信息（扣1分）

“维护信息/数据”：维护什么信息/数据（扣1分）

(2) 用例体现系统需要完成的功能

用例描述了系统需要完成的功能，因此不是系统需要处理的就不是用例；

如问题陈述中明确表示对员工的奖惩不在本系统中处理，因此有关这些的用例都是错误的（扣1分）

(3) 用例粒度

用例是一组用例实例，太细陷入了功能分解，用例文档不好写，后面的用例分析更没法做。

如计算工资，没有必要把三类员工分别作为单独的用例，因为其操作流程比较简单且类似（扣0.5-1分）。

(4) 用例关系的使用

扩展和包含关系从本质上来说非常类似，因此在某些情况下可以混用，但一定要注意这两种关系的方向是不同的，扩展是子用例对主用例的扩展，而包含则表示主用例包含子用例，（扩展方向错误，扣0.5-1分）

一般情况下，本系统不需要使用任何用例关系，但如果有人使用，并且使用正确则酌情加1-2分；可以使用的关系有：“计算工资”可以分解出“产生确认消息”的子用例（建议用包含），项目经理的“管理员工考勤”可以分解出“修改考勤”、“修改订单”（建议用扩展）。

最后，用例之间只存在这三种关系，使用任何其它不正确的关系都是错误的（扣0.5-1分）

3.2 用例文档部分

用例文档的编写可以按照给定的模板，也可以不按给定的模板；如果使用其它模板，则要求必须包含的项目：用例名称、用例描述、前置条件、基本路径、备选路径、非功能需求等项目。

用例文档书写时必须遵循一定的格式，条理清楚，如果没有格式，内容混乱，酌情扣1-2分。

(1) 用例文档中的基本部分

用例名：必须与用例图中的名称一致，如用例图中为“查看工资”，用例文档中为“查看工资信息”，这种不一致扣1分；

参与者：参与者部分只与该用例相关的参与者，需要与用例图中保持一致（扣0.5-1分）

涉众利益：此部分主要是写清楚相关的涉众即可，至于涉众利益的具体描述不必深究。

(2) 前置条件和后置条件

前置条件约束是约束用例开始前系统状态，后置条件约束用例执行后：系统状态前置、后置条件必须是系统能检测到的，而且对于前置条件必须在用例执行前可以检测到（扣1分）。

后置条件一般可以不写。

用例编号: UC01 → 用例名: 查看工资信息 → → → 作者: 王 斌
用例描述: 公司员工利用本用例来查看本人的工资信息
执行者
小时工, 普通员工, 销售人员, 项目经理以及项目管理数据库
相关用例 (指在用例图中存在关系的用例)
记录工作时间, 记录定单, 确认工资信息
前置条件 (本用例的前置条件, 指用例开始前可以检测的内容)
1. 小时工, 普通员工, 销售人员和项目经理正确登录到该系统 (一定要描述清楚, 按您的理解四类员工必须同时登录?)
2. 系统已经产生员工的工资支付信息
3. 项目经理已经对本月的工资进行了确认

(3) 基本路径的编写

用例的基本路径代表了本用例所要达到的主要目标;以参与者与系统进行交互的方式进行描述;编写时除第一句话,其他语句一般都以参与者或用例作为主语。下面的一些典型错误酌情扣 0.5-1 分

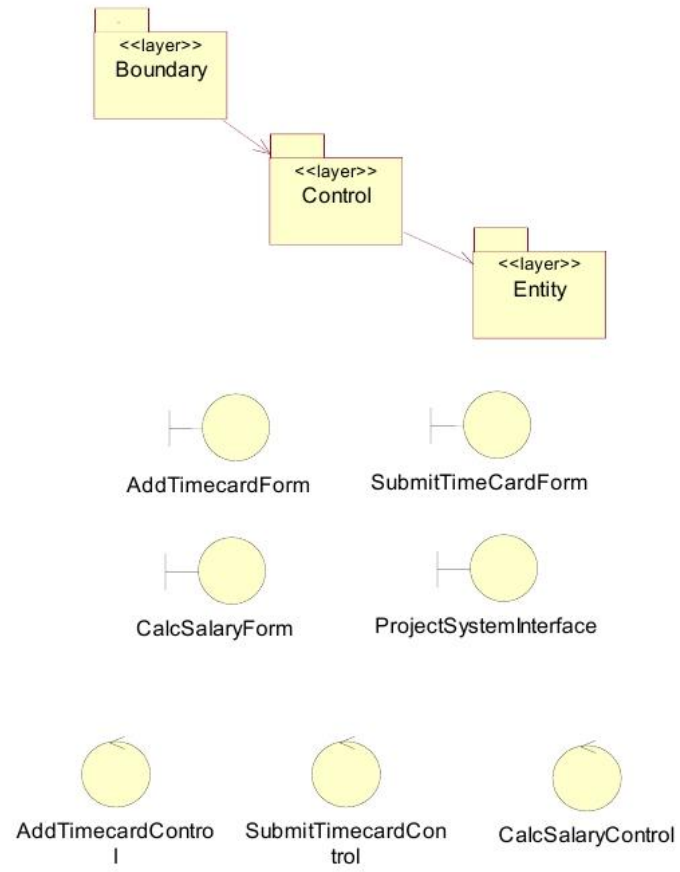
基本路径只能处理一种情况,不能把所有的工作都罗列在一起,路径中的每个步骤必须是系统需要且能够处理的。如计算工资,由于小时工、普通员工、销售人员计算方法并不相同,因此该用例的基本路径只能处理一种情况,其他情况应该作为备选路径。(扣 1 分)

用例编号: → UC01 → 用例名: → 维护个人考勤信息 → → → 作者: 王 斌
用例描述: 本用例容许员工维护个人的考勤信息。对普通员工和小时工可以编辑和查看考勤信息,对销售人员不响应编辑考勤信息操作。项目数据库系统提供员工信息表。用例规约得分: 6 分
基本路径
1. 用例起始于员工利用该系统进行维护考勤信息 (基本路径中需要明确具体的维护内容,是新增还是修改还是查看?)
2. 员工成功登录系统。(这是基本路径吗?)
3. 员工进行考勤信息的维护操作。(如何操作?)
4. 系统保存员工的考勤信息。

基本路径
1.用例起始于周末或月末时间发起一个计算工资请求。
2.时间执行者按照规定的要求,向系统输入所需信息(时间执行者除了启动该用例之外就做不到任何事情了!)
3.系统根据得到的信息查找需要计算工资的雇员记录,并据此向原有项目数据库发出请求。
4.系统得到原有项目数据库返回的所需信息。
5.系统开始计算那些应该计算工资的雇员的工资。
6.系统记录每一笔工资结果。
7.系统执行所需的其它必要操作。(什么操作,用例文档表述需求,需求一定要表述清楚)
8.最后,系统执行向项目经理提示的子用例操作。(?)

Flow of Events: (是一个顺序的流程,不是一堆工作的罗列)
1. 用例起始于员工填写个人用户名和密码,登录系统之后 (这是用例的起始?不是前置条件吗?)
2. 员工查询本月(普通工和销售人员)或者本周(小时工)工资。
3. 员工可以查询之前的工资信息。
6. 系统提示员工关闭当前系统页面。
6. 员工关闭系统。

备选构架，以及各层中所包含的类：



(4) 备选路径

备选路径的编写比较自由，主要注意的问题是基本路径的衔接（即是从那个基本路径中派生出来的，一般是通过编号来表示），这部分不必求全（可能有些情况没有考虑到），写了并且正确即可。

(5) 用例关系在用例文档中的使用

用例图中存在关系时，需要在用例文档中进行体现，此部分如果有人做，并且做得正确则加 1-2 分，做错了就要进行相应的扣分。

扩展关系：在主用例中定义“扩展点”（增加“扩展点”项），在用例流程中需要扩展的地方插入扩展点（格式一般为：<扩展点名>）。在子用例中声明其父用例和对应的扩展点。

包含关系：在主用例中定义“包含的子用例”（增加“包含的子用例”项），在用例流程中直接写“调用/包含***子用例”。

二、第五章作业（用例分析）

1. 评分档次

一档（10 分）：图形（文本）条理清楚，无任何错误

二档（8-9 分）：图形/文本清楚，存在个别错误

三档（6-7 分）：图形/文本一般，存在一定的错误

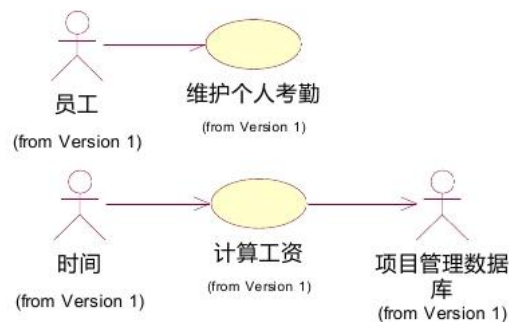
四档（5 分）：图形/文本条理不清，存在致命错误或错误数过多

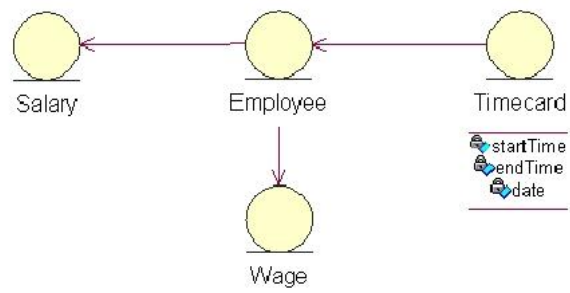
一般情况下按错别个数扣分，每个错误按严重程度扣 0.5、1、2 分，最终成绩向上取整；同类错误不重复扣分。

2. 参考答案

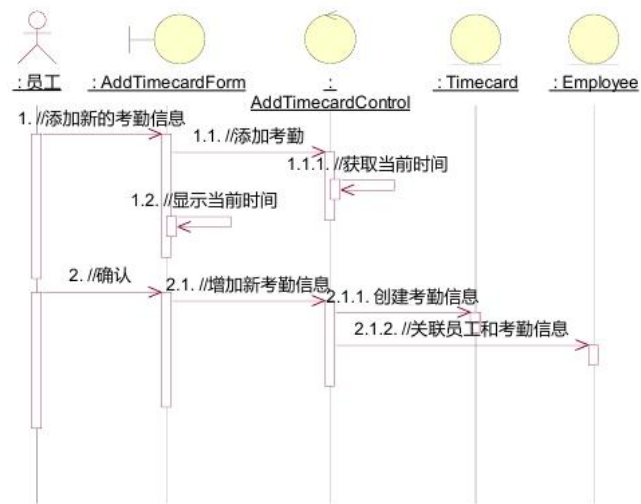
作业答案部分仅供参考，学生的作业可能会多种多样，具体按照第三部分的典型错误扣分（相关模型参加 PayrollModel(Analysis).mdl 文件）：

第一个迭代周期（其中维护个人考勤只做基本路径，而计算工资可以只考虑最简单的情况-即计算临时工工资；此外，可以考虑把销售人员“维护个人订单”用例加入）

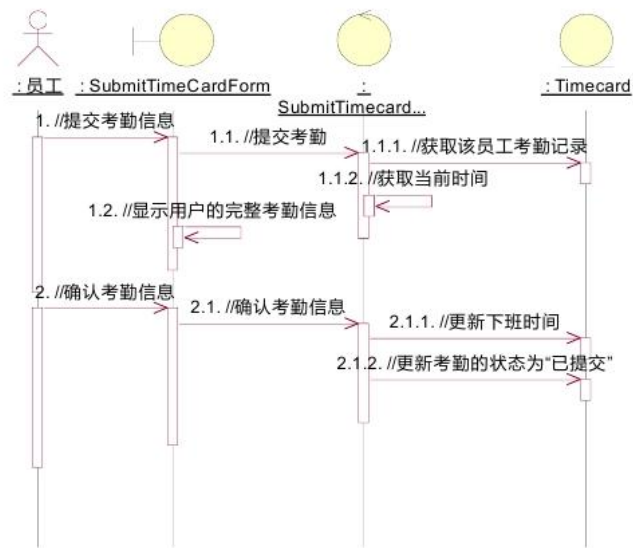




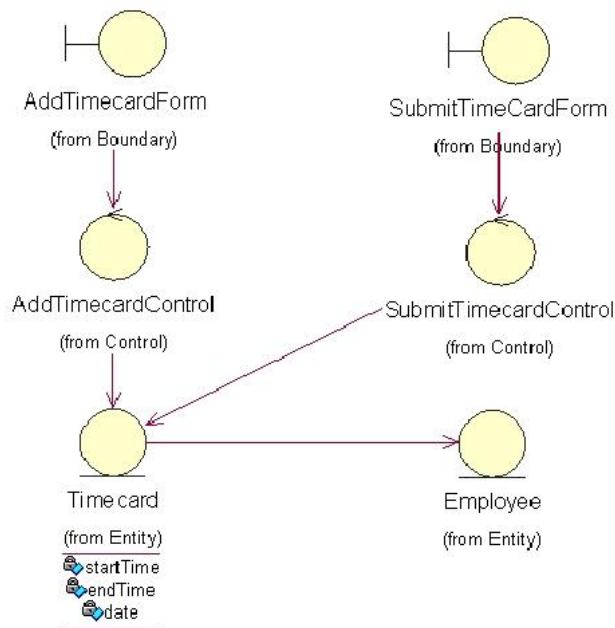
维护个人考勤（基本路径）-添加新考勤的顺序图



维护个人考勤（基本路径）-提交考勤的顺序图



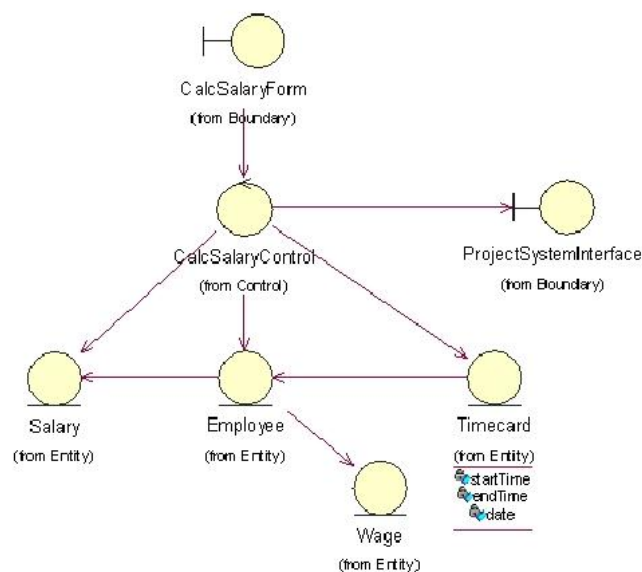
维护个人考勤（基本路径）-VOPC 类图



计算工资-计算临时工工资顺序图



计算工资-VOPC 类图



3. 典型错误情况

3.1 构架部分

(1) 迭代周期的选取

重点考虑对系统构架有重要影响的用例（即带来重要的关键抽象（即核心实体类）的用例），同时对于复杂的用例可适当简化。

参考答案中给出的是维护个人考勤和计算工资两个用例，同时还可以考虑增加销售人员维护个人订单用例。此外对这些用例可作适当的简化，只考虑其关键路径（基本路径或一些核心路径）即可。选择依据：

- ⊙ 维护个人考勤（或登记考勤）用例包含核心概念“考勤”，必须有；否则扣1分；
- ⊙ 计算工资用例包含核心概念“工资”、“工资级别”，此外该用例还与外部系统“项目管理数据库”进行交互，这也是一个非常重要的分析要素，应该有；否则扣0.5-1分；
- ⊙ 维护个人订单用例带来核心概念“订单”，可以有；但由于在本考勤系统中，订单的处理模式和考勤类似，对系统高层构架影响不大，可以没有。

(2) 模型组织

模型组织是本次作业考核的一个非常重要的项目，本次作业要求提交 Word 文档，主要考察是否有这些内容，而且这些内容组织比较清楚：

- ⊙ 必须利用包图定义系统分层构架，并按照分层的思想说明所找到的类放在不同的包

中：如果类全部放在根视图或一个包内则扣 1 分；

(3) 边界类和控制类

这个不是重点，按照课上所讲的原则即可，同时可灵活应用；参考答案中对于“维护个人考勤（基本路径）”这个用例定义了两个控制类，这时因为按照用例规约，登记考勤本身就分为两个独立的阶段（添加新考勤、提交考勤），不同的阶段定义了不同的控制类。

(4) 实体类的完备性

这也是一个考察重点，在本系统中，根据第一个迭代周期定义的实体类有：考勤、员工、工资、工资级别（、订单）等这些重要的类，如果缺少考勤、工资、工资级别等关键实体类，则扣 1 分；当然，在本系统中，还可以定义其它的一些实体类，如可以将员工分为临时工、普通员工、销售人员，这样他们与员工类之间存在泛化关系。

此外，还有一些典型的错误，如把“系统”、“数据库”这样的要素作为实体类，出现这样的常识性错误扣 1 分。

3.2 用例分析部分

(1) 顺序图的基本画法

顺序图放置在相应的用例实现里面，对于用例不同的路径画不同的顺序图；

顺序图中的对象应该都对于前面所找出来的类，出现没有类的对象则扣 1 分；

(2) 顺序图的合理性

顺序图的消息对应的是接受该消息的类的职责，因此必须保证接受消息的类能够实现所要求的职责，如您不能要求一个“提交考勤控制类”提供考勤信息，这些信息应该由“考勤”实体类提供；此外，顺序图中的消息必须明确、可行，要和用例文档中的约定一致，不能过于抽象；出现不可行或不合理的信息，则扣 1 分。

(3) VOPC 图

VOPC 图是某个用例实现的类图，应放在用例实现里面

VOPC 类图中的类是那些出现在该用例所有顺序图中的类

类之间定义关联关系，关系的依据主要是顺序图中的消息（A 向 B 发消息，则存在由 A 指向 B 的关联，注意关联的方向）和现实业务中的概念（如考勤信息与员工之间存在关系，表明这是哪个员工的考勤，这些是业务知识）。缺少关联或关联的方向不对则扣 1 分。

三、第八章作业（架构设计）

1. 评分档次

此部分按照满分 5 分计算，以此作为评分标准，基本的评分准则如下：

一档（5 分）：图形（文本）条理清楚，无任何错误

二档（4 分）：图形/文本清楚，存在个别错误

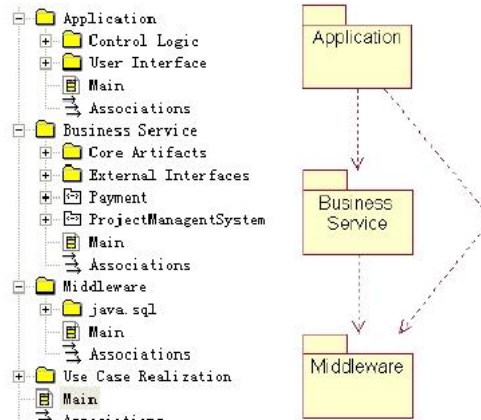
三档（3 分）：图形/文本一般，存在一定的错误

四档（2 分）：图形/文本条理不清，存在致命错误或错误数过多

一般情况下按错别个数扣分，每个错误按严重程度扣 0.5、1、2 分，最终成绩向上取整；同类错误不重复扣分。

2. 参考答案

软件构架不是一步到位的，随着设计的深化，构架的内容得到不断的完善和扩充，构架设计的主要思路是建立软件的合理层次结构，提高设计的质量。因此本部分的评分比较灵活，只要有合理的分层思想而不是简单的 B-C-E 模式即可。本系统提供的参考构架：



3. 典型错误情况

构架设计的主要思路是建立软件的合理层次结构，提高设计的质量。因此本部分的评分比较灵活，只要有合理的分层思想而不是简单的 B-C-E 模式即可。具体的评分点：

- ⊙ 直接套用 B-C-E 模式，则扣 0.5 分；
- ⊙ 没有结构，所有元素均放在一层，或分层思想模糊，没有明确的标准，则扣 1 分。

四、第九章作业（用例设计）

1. 评分档次

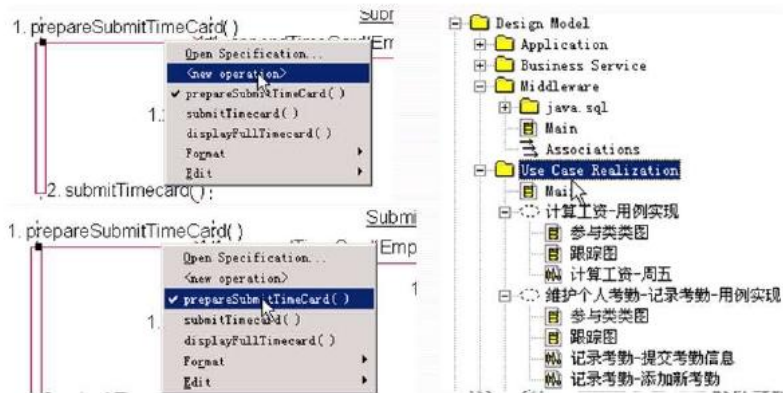
此部分按照满分 10 分计算，以此作为评分标准，基本的评分准则如下：

- 一档（10 分）：图形（文本）条理清楚，无任何错误
 - 二档（8-9 分）：图形/文本清楚，存在个别错误
 - 三档（6-7 分）：图形/文本一般，存在一定的错误
 - 四档（5 分）：图形/文本条理不清，存在致命错误或错误数过多
- 一般情况下按错别个数扣分，每个错误按严重程度扣 0.5、1、2 分，最终成绩向上取整；同类错误不重复扣分。

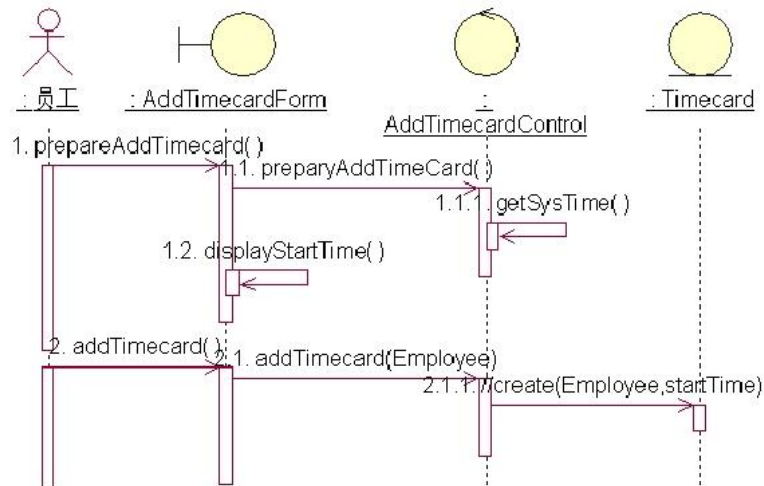
2. 参考答案

用例设计的主要工作是：用设计元素取代分析类、将职责细化为类的操作，其组织方式

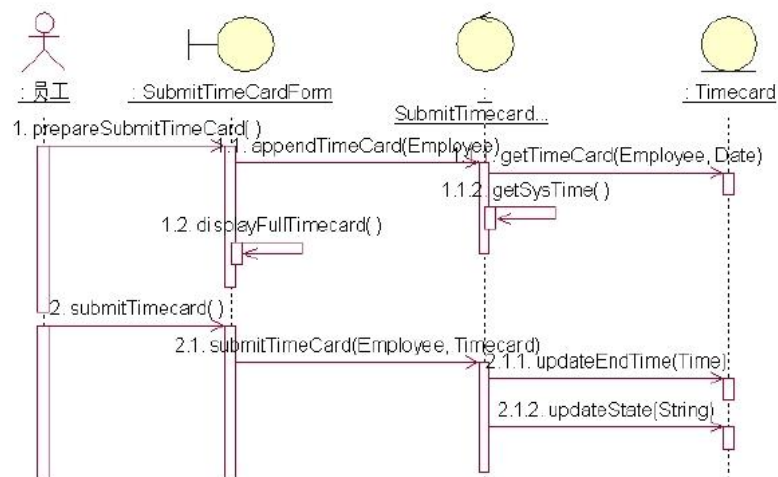
依然是按照用例实现来完成。如下图所示：



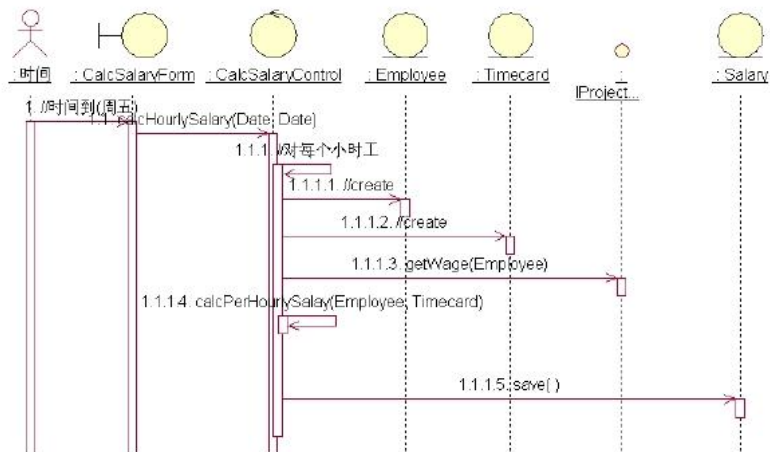
“记录考勤-添加新考勤”顺序图



“记录考勤-提交考勤”顺序图

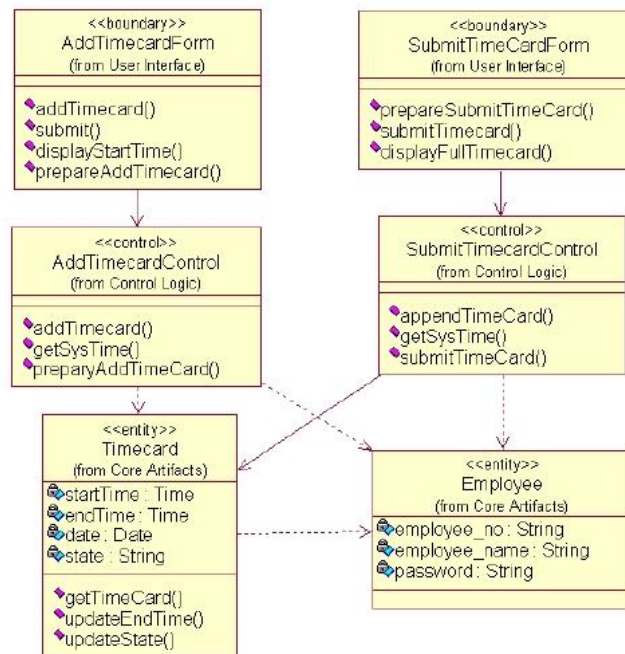


“计算工资”顺序图（周五-计算临时工工资）

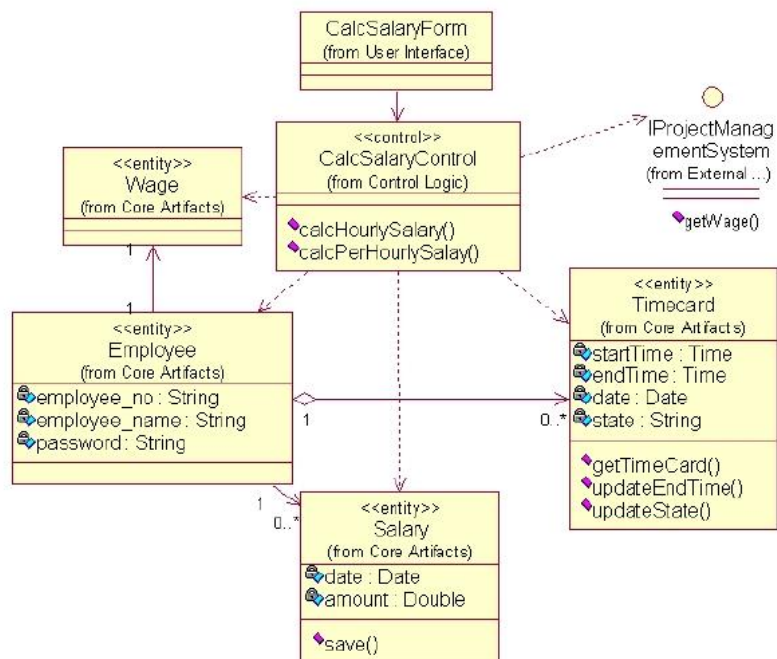


类图设计的主要工作包括：添加类必要的属性、完善类操作的定义、完善类关系的定义（如依赖、聚合等）

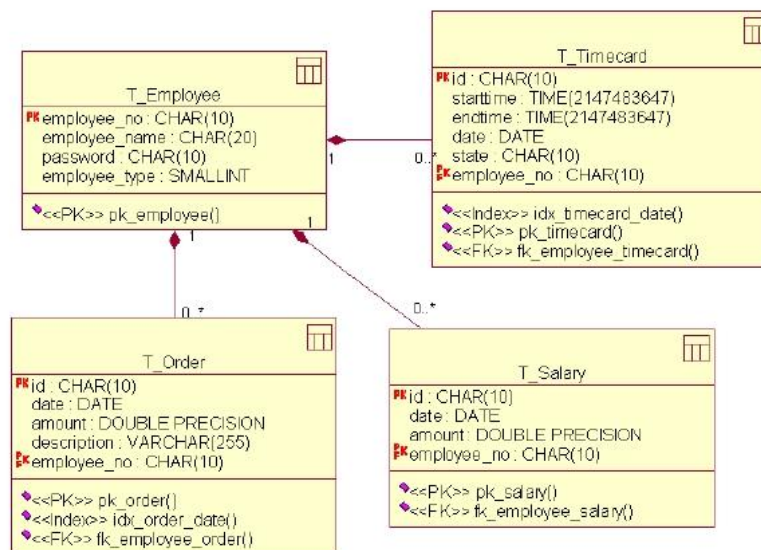
“记录考勤”VOPC 类图



“计算工资” VOPC 类图



数据库设计:



3. 典型错误情况

3.1 用例设计部分

(1) 用例设计-模型组织

模型的组织方式与用例分析时类似，利用软件构架和用例实现来分层组织，如果没有按照这种思路去组织（各种类放在构架包中，交互图、VOPC 图则按照用例实现来组织），则扣 1-2 分。

(2) 用例设计-交互图设计

用例设计的交互图的完成思路可以与用例分析时类似，但此时必须保证所有的消息（除非用“//”注释了该消息）都由相应的目标类的操作来支持。具体的扣分值：

- ⊙ 交互图中的对象不属于任何类，扣 1 分
- ⊙ 交互图中的消息不能由目标类的操作来支持，扣 1 分

(3) 类设计

VOPC 图是某个用例实现的类图，应放在用例实现里面；VOPC 类图中的类是那些出现在该用例所有顺序图中的类。具体的扣分值：

- ⊙ 类本身应该有较完备的属性和操作，缺少必要属性或操作的扣 0.5 分
- ⊙ 类之间的关系要较清楚地定义，注意导航性、依赖、聚合等关系的适当使用，应用合理的适当可加 0.5-1 分，而存在错误或没有定义任何导航性，则扣 0.5 分。

3.2 数据库设计

数据库设计比较简单，可以有系统生成，也可从头开始定义，具体的注意实现有：

- ⊙ 每个数据表应该都有一个主键，否则扣 0.5-1 分
- ⊙ 数据库模型中注意正确的使用外键（FK）来表示表之间的关系，否则扣 0.5-1 分
- ⊙ 利用自动生成后一定要注意修改字段的类型为合适的类型，否则扣 1 分